

UNIVERSIDAD DEL VALLE DE GUATEMALA



Proyecto 2

Andre Marroquin Tarot - 22266

Sergio Orellana - 221122

Computación paralela y distribuida

Parte A:

DES Proceso de cifrado y descifrado

Datos de entrada (tamaños)

- **Bloque de datos:** 64 bits.
- **Clave:** 64 bits (56 útiles + 8 de paridad).
- **Subclaves:** 16 subclaves $K_1..K_{16}K_1..K_{16}K_1..K_{16}$ de 48 bits.

Glosario

- **PC-1:** Elección Permutada 1. Quita paridad y reordena la clave (64→56).
- **C0, D0:** Mitades de 28 bits que resultan de PC-1 (izq./der.).
- **PC-2:** Elección Permutada 2. Selecciona/reordena 48 bits de $C_i \parallel D_i \rightarrow K_i$
- **IP:** Permutación inicial al bloque de datos.
- **IP⁻¹:** Inversa de IP (permuta final).
- **L0, R0 / Li, Ri:** Mitades izquierda/derecha del bloque tras IP y tras la ronda i.
- **E(R):** Expansión de 32→48 bits para poder XORear con $K_iK_{-i}K_i$.
- **S-Boxes:** 8 tablas que convierten 6→4 bits (en total 48→32).
- **P:** Permutación que reordena los 32 bits que salen de las S-Boxes.
- **f(R,K):** Función de ronda = $P(S(E(R) \oplus K))$.
- $\oplus$: XOR (o exclusivo).
- $\parallel$: Concatenación de bits.

Cifrado (plaintext → ciphertext)

1. **Key schedule**
 - 1.1 PC-1: clave 64→56; dividir en C0 y D0 (28/28).
 - 1.2 Para $i=1..16$: rotar C_{i-1}, D_{i-1} (1 o 2 bits) → C_i, D_i .
 - 1.3 PC-2 sobre $C_i \parallel D_i \rightarrow K_i$ (48 bits).
2. **IP** al bloque de 64 bits.
3. **Dividir** en L0 y R0 (32/32).
4. **Rondas Feistel (i=1..16):**
 - 4.1 $X = E(R_{i-1}) \oplus K_i$ (32 → 48, XOR) .
 - 4.2 Pasar X por S-Boxes → 32 bits.

4.3 P sobre esos 32 bits.

4.4 **Actualizar mitades:**

- $L_i = R_{i-1}$

- $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$.

5. **Intercambio final:** formar $R_{16} || L_{16}$.

6. $IP^{-1} \rightarrow$ bloque cifrado de 64 bits.

Descifrado (ciphertext $\rightarrow$ plaintext)

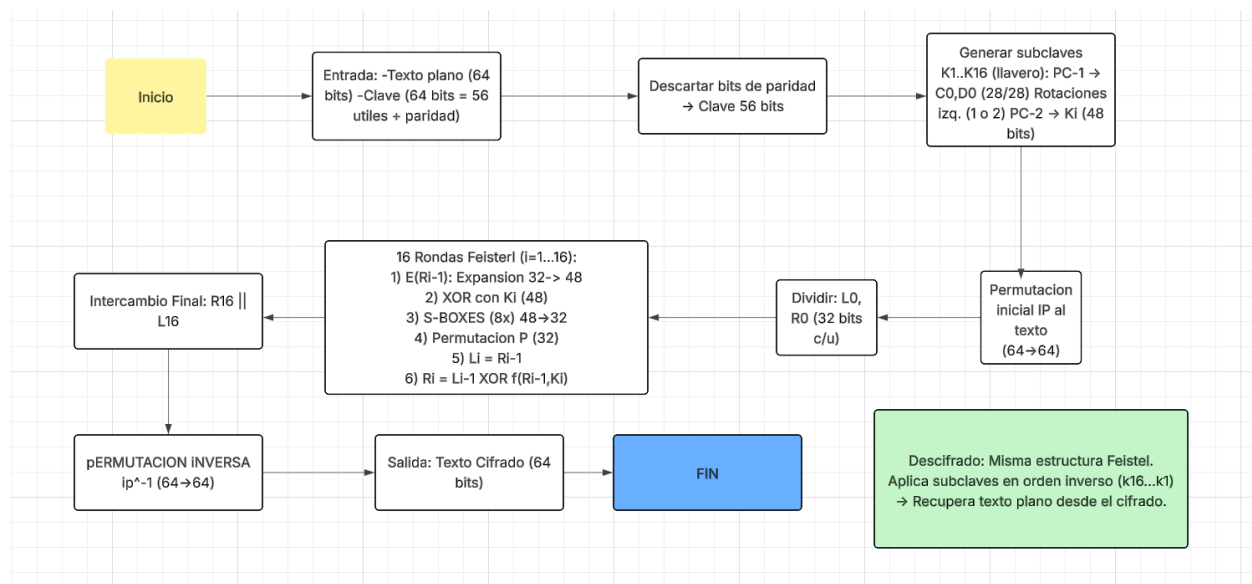
1. IP al bloque cifrado; dividir en L_0, R_0 .

2. **Rondas Feistel ($i=16..1$):** repetir los pasos 4.1–4.4 pero usando las subclaves en orden **inverso**: $K_{16}, K_{15}, \dots, K_1$.

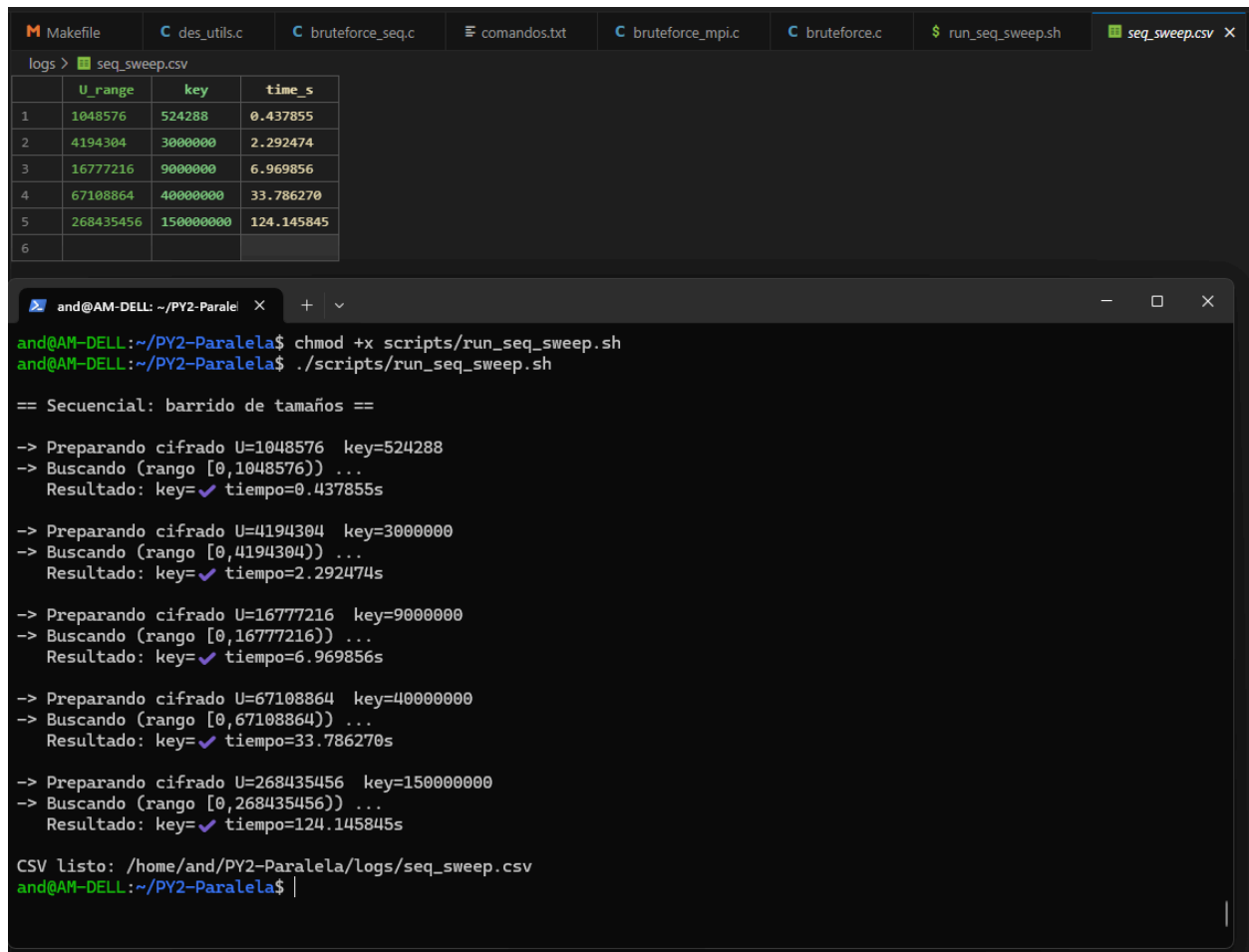
3. **Intercambio final:** $R_{16} || L_{16}$.

4. $IP^{-1} \rightarrow$ texto plano original.

Diagrama de Flujo



Inciso 3



```
logs > seq_sweep.csv
```

	U_range	key	time s
1	1048576	524288	0.437855
2	4194304	3000000	2.292474
3	16777216	9000000	6.969856
4	67108864	40000000	33.786270
5	268435456	150000000	124.145845
6			

```
and@AM-DELL: ~/PY2-Paralela$ chmod +x scripts/run_seq_sweep.sh
and@AM-DELL:~/PY2-Paralela$ ./scripts/run_seq_sweep.sh

== Secuencial: barrido de tamaños ==

-> Preparando cifrado U=1048576 key=524288
-> Buscando (rango [0,1048576)) ...
Resultado: key=✓ tiempo=0.437855s

-> Preparando cifrado U=4194304 key=3000000
-> Buscando (rango [0,4194304)) ...
Resultado: key=✓ tiempo=2.292474s

-> Preparando cifrado U=16777216 key=9000000
-> Buscando (rango [0,16777216)) ...
Resultado: key=✓ tiempo=6.969856s

-> Preparando cifrado U=67108864 key=40000000
-> Buscando (rango [0,67108864)) ...
Resultado: key=✓ tiempo=33.786270s

-> Preparando cifrado U=268435456 key=150000000
-> Buscando (rango [0,268435456)) ...
Resultado: key=✓ tiempo=124.145845s

CSV listo: /home/and/PY2-Paralela/logs/seq_sweep.csv
and@AM-DELL:~/PY2-Paralela$ |
```

En las pruebas secuenciales, al aumentar el tamaño del espacio de llaves (U_range) el tiempo de búsqueda creció prácticamente de forma lineal, tal como esperaría un ataque de fuerza bruta clásico. Por ejemplo, al aumentar el rango varias veces, el tiempo se multiplicó entre $\sim 3\times$ y $\sim 5\times$ por variaciones normales del sistema, pero el rendimiento por segundo se mantuvo muy estable alrededor de 1.2–1.3 millones de llaves por segundo (ejemplo 524 288 llaves en 0.438 s $\rightarrow \approx 1.20$ M/s; 3 000 000 en 2.29 s $\rightarrow \approx 1.31$ M/s; 150 000 000 en 124.1 s $\rightarrow \approx 1.21$ M/s). Dado que las llaves probadas están cerca de la mitad del espacio, el número promedio de intentos es $\approx U/2$, por lo que el tiempo total escala con U.

Inciso 4:

Se hizo funcionar el bruteforce.c

```

and@AM-DELL: ~/PY2-Paralela$ mpirun -np 4 ./bin/bruteforce_orig -s "the" -L 0 -U 16777216
=====
BruteDES • MPI (archivo original adaptado)
- Tabla por proceso + resumen global
=====

→ BRUTEFORCE MPI (original adaptado)
• Procesos : 4
• Subcadena: "the"
• Rango : [0, 16777216)

• Detalle por proceso
RANK | START | END | TESTS | STATUS | TIME(s)
-----|-----|-----|-----|-----|-----
0 | 0 | 4194304 | 105735 | FOUND | 0.0650
1 | 4194304 | 8388608 | 103410 | STOP(SIGNAL) | 0.0651
2 | 8388608 | 12582912 | 105394 | STOP(SIGNAL) | 0.0651
3 | 12582912 | 16777216 | 82531 | STOP(SIGNAL) | 0.0650

• Resultado: ✓ Llave encontrada
- Rank : 0
- Llave : 105734
- Texto : Save the planet

• Resumen global
- Llaves probadas totales : 397070
- Tiempo total (max rank) : 0.065147 s

and@AM-DELL:~/PY2-Paralela$

```

Clave encontrada "Save the planet "

Inciso 5:

a) decrypt(key, *ciph, len) y encrypt(key, *ciph, len)

Están sobre el backend des_compat_* (OpenSSL).

- **Entrada:** key entero con 56 bits efectivos, ciph buffer binario, len múltiplo de 8.
- **Salida:** escriben el resultado en el mismo ciph, usando un buffer temporal para evitar aliasing.

Diagrama:

Key (unit64) → 8 bytes + Paridad DES → DES-Key-Schedule
 Ciph → [bloques de 8 B] → DES-ECB(enc/dec) → tmp → copia a Ciph

b) try_key(key, *ciph, len)

Prueba una clave candidata y decidir si sirve buscando una marca en el texto plano. En este caso la marca es `search_str = " the "`.

Flujo

1. Clona el cifrado a un buffer temporal.
2. Descifra con `decrypt(key, temp, len)`.
3. Convierte el buffer a C-string poniendo `temp[len]=0`.
4. Busca la subcadena con `strstr`.
5. Devuelve 1 si hay match; 0 si no.

Diagrama:

Key $\rightarrow$ `decrypt (Key, ciph)` $\rightarrow$ `temp (texto)`
Temp $\rightarrow$ `strstr (temp, search_str)` $\rightarrow$ ¿encontrado?

c) `memcpy`

Copia exactamente `n` bytes de `src` a `dst` sin preocuparse por el contenido. Es la herramienta correcta cuando no hay solapamiento.

Implementacion

- Copiar cifrado $\rightarrow$ temporal antes de descifrar (`try_key`).
- Copiar bloques de 8B dentro del backend DES.
- Copiar `tmp` $\rightarrow$ `ciph` para simular in-place seguro.

Diagrama:

`memcpy(dst, src, n)`

Si existiera solapamiento se usa `memmove`.

d) `strstr`

Busca la primera aparición de una subcadena en una cadena C.
Retorna puntero a la posición o NULL si no la encuentra.

Es la verificación de que la clave fue correcta: al descifrar, debe aparecer search_str dentro del texto plano.

Diagrama:

```
return strstr(temp, search_str) != NULL;
```

Iniciso 6:

Primitivas MPI:

bruteforce.c reparte el espacio de claves entre n_nodes procesos. Cada proceso prueba su rango y, si acierta, avisa a todos incluido él mismo para que el maestro imprima.

a) MPI_Irecv

Recepción no bloqueante. Inicia la recepción y vuelve de inmediato.

Todos los ranks postean un irecv para estar listos a recibir la clave cuando cualquiera la encuentre.

Codigo:

```
long recv_key = 0;  
MPI_Irecv(&recv_key, 1, MPI_LONG, MPI_ANY_SOURCE, 0, comm, &req);
```

b) MPI_Send

Envío bloqueante. Retorna cuando el runtime ya puede garantizar la entrega buffering o receptor listo.

Patron cuando un proceso encuentra la clave...

Codigo:

```
for (int node = 0; node < n_nodes; ++node) {  
    MPI_Send(&found, 1, MPI_LONG, node, 0, MPI_COMM_WORLD);  
}
```

c) MPI_Wait

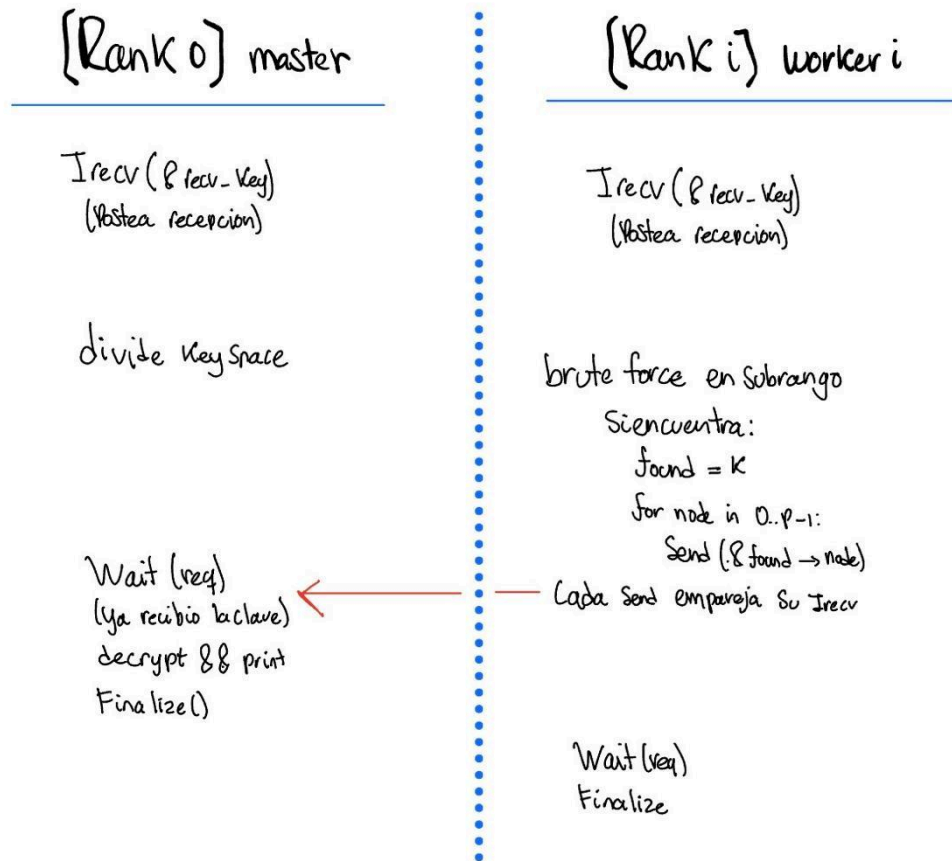
Bloquea hasta que una operación no bloqueante asociada a MPI_Request finalice.

Completar el irecv en todos los ranks antes de finalizar.

Codigo:

```
MPI_Wait(&req, &st);  
if (id == 0) {  
    Imprimir el resultado con la key aca  
}
```

Flujo comunicacion:



Pseudocodigo flujo:

```

MPI_Irecv(&recv_key, 1, MPI_LONG, ANY_SOURCE, 0, comm, &req);

if (encontre) {
    for (node=0..n_nodes-1) MPI_Send(&found, 1, MPI_LONG, node, 0, comm);
}

MPI_Wait(&req, &st);

```

Parte B:

1)

Evidencias:

Texto:

```
Makefile  des_utils.c  bruteforce_seq.c  comandos.txt  seq_sweep.csv  bruteforce_mpi.c  bruteforce.c  run_seq_swee
data > mensaje.txt
1 Solo a Néstor, a Nación Nueva, me llamo la atención cómo la Ó brillaba en el nombre y quedaba grabada en mi memoria.
```

Encriptacion:

```
and@AM-DELL: ~/PY2-Paralela$ ./bin/bruteforce_seq --encrypt -i data/mensaje.txt -k 123456 -o data/cipher.bin
=====
BruteDES • Secuencial
- Cifra y ataca DES-ECB (OpenSSL EVP)
- Rango de llaves configurable [L, U)
=====

✓Cifrado generado
• Entrada : data/mensaje.txt (bytes=121)
• Salida : data/cipher.bin (bytes=128, padded x8)
• Llave : 123456
and@AM-DELL:~/PY2-Paralela$ |
```

Secuencial:

```
and@AM-DELL: ~/PY2-Paralela$ ./bin/bruteforce_seq --bruteforce -c data/cipher.bin -s "me llamo" -L 0 -U 16777216
=====
BruteDES • Secuencial
- Cifra y ataca DES-ECB (OpenSSL EVP)
- Rango de llaves configurable [L, U)
=====

→ BRUTEFORCE SECUENCIAL
• Archivo : data/cipher.bin (bytes=128)
• Subcadena: "me llamo"
• Rango : [0, 16777216)
• Resultado: ✓Llave encontrada
- Llave : 57920
- Texto : Solo a Néstor, a Nación Nueva, me llamo la atención cómo la Ó brillaba en el nombre y quedaba grabada en
mi memoria.
• Tiempo : 0.096546 s
and@AM-DELL:~/PY2-Paralela$ |
```

Paralelo:

```
and@AM-DELL: ~/PY2-Paralela
• Tiempo : 0.096546 s
and@AM-DELL:~/PY2-Paralela$
and@AM-DELL:~/PY2-Paralela$ mpirun -np 4 ./bin/bruteforce_mpi -c data/cipher.bin -s "me llamo" -L 0 -U 16777216
=====
BruteDES • MPI
- División equitativa del rango y early-stop
=====

→ BRUTEFORCE MPI
• Procesos : 4
• Archivo : data/cipher.bin (bytes=128)
• Subcadena: "me llamo"
• Rango : [0, 16777216)

• Detalle por proceso
RANK | START | END | TESTS | STATUS | TIME(s)
-----|-----|-----|-----|-----|-----
0 | 0 | 4194304 | 57921 | FOUND | 0.0809
1 | 4194304 | 8388608 | 56097 | STOP(SIGNAL) | 0.0788
2 | 8388608 | 12582912 | 57298 | STOP(SIGNAL) | 0.0806
3 | 12582912 | 16777216 | 55265 | STOP(SIGNAL) | 0.0788

• Resultado: ✓ Llave encontrada
- Llave : 57920
- Texto : Solo a Néstor, a Nación Nueva, me llamo la atención cómo la Ó brillaba en el nombre y quedaba grabada en mi memoria.

• Resumen global
- Llaves probadas totales : 226581
- Tiempo total (max rank) : 0.080895 s

and@AM-DELL:~/PY2-Paralela$
```

2)

a) Medir tiempo usando la llave 123456

Encriptacion:

```
and@AM-DELL:~/PY2-Paralela$ ./bin/bruteforce_mpi -s ./data/cipher.bin -c ./des_encrypt -e encrypt
and@AM-DELL:~/PY2-Paralela$ ./bin/bruteforce_seq --encrypt -i data/mensaje.txt -k 123456 -o data/c_123456.bin
=====
BruteDES • Secuencial
- Cifra y ataca DES-ECB (OpenSSL EVP)
- Rango de llaves configurable [L, U)
=====

✓ Cifrado generado
• Entrada : data/mensaje.txt (bytes=32)
• Salida : data/c_123456.bin (bytes=32, padded x8)
• Llave : 123456 (efectiva dec=57920, hex=0x000000000000e240)
```

Solucion:

```

and@AM-DELL:~/PY2-Paralela$ mpirun -np 4 ./bin/bruteforce_mpi -c data/c_123456.bin -s "es una prueba de" -L 0 -U 16777216
6
=====
BruteDES • MPI
- División equitativa del rango y early-stop
=====

→ BRUTEFORCE MPI
• Procesos : 4
• Archivo : data/c_123456.bin (bytes=32)
• Subcadena: "es una prueba de"
• Rango : [0, 16777216)

• Detalle por proceso


| RANK | START    | END      | TESTS | STATUS       | TIME(s) |
|------|----------|----------|-------|--------------|---------|
| 0    | 0        | 4194304  | 57921 | FOUND        | 0.0367  |
| 1    | 4194304  | 8388608  | 59551 | STOP(SIGNAL) | 0.0367  |
| 2    | 8388608  | 12582912 | 59446 | STOP(SIGNAL) | 0.0367  |
| 3    | 12582912 | 16777216 | 58431 | STOP(SIGNAL) | 0.0368  |



• Resultado: ✓Llave encontrada
- Rank : 0
- Llave : 57920 (efectiva dec=57920, hex=0x000000000000e240)
- Texto : Esta es una prueba de proyecto 2

• Resumen global
- Llaves probadas totales : 235349
- Tiempo total (max rank) : 0.036754 s

```

b) Medir tiempo usando la llave 18014398509481983

Encriptacion:

```

and@AM-DELL: ~/PY2-Paralela$ ./bin/bruteforce_seq --encrypt \
-i data/mensaje.txt \
-k 18014398509481983 \
-o data/cipher_b.bin
=====
BruteDES • Secuencial
- Cifra y ataca DES-ECB (OpenSSL EVP)
- Rango de llaves configurable [L, U)
=====

✓Cifrado generado
• Entrada : data/mensaje.txt (bytes=32)
• Salida : data/cipher_b.bin (bytes=32, padded x8)
• Llave : 18014398509481983 (efectiva dec=17731819709333246, hex=0x003efefefefefefe)
and@AM-DELL:~/PY2-Paralela$ |

```

Solucion:

```
and@AM-DELL:~/PY2-Paralela$ mpirun -np 4 ./bin/bruteforce_mpi \  
-c data/cipher_b.bin \  
-s "es una prueba de" \  
-L 0 -U 72057594037927936 | tee logs/mpi_case_b_full.txt
```

```
=====
```

BruteDES • MPI

- División equitativa del rango y early-stop

```
=====
```

c) Medir tiempo usando la llave 18014398509481984

Encriptacion:

```
and@AM-DELL: ~/PY2-Paralela$ ./bin/bruteforce_seq --encrypt \  
-i data/mensaje.txt \  
-k 18014398509481984 \  
-o data/cipher_c.bin
```

```
=====
```

BruteDES • Secuencial

- Cifra y ataca DES-ECB (OpenSSL EVP)
- Rango de llaves configurable [L, U)

```
=====
```

✓Cifrado generado

- Entrada : data/mensaje.txt (bytes=32)
- Salida : data/cipher_c.bin (bytes=32, padded x8)
- Llave : 18014398509481984 (efectiva dec=18014398509481984, hex=0x0040000000000000)

```
and@AM-DELL:~/PY2-Paralela$ |
```

Solucion:

```

and@AM-DELL:~/PY2-Paralela$ mpirun -np 4 ./bin/bruteforce_mpi \
-c data/cipher_c.bin \
-s "es una prueba de" \
-L 0 -U 72057594037927936 | tee logs/mpi_case_c_full.txt
=====
BruteDES • MPI
- División equitativa del rango y early-stop
=====

→ BRUTEFORCE MPI
• Procesos : 4
• Archivo : data/cipher_c.bin (bytes=32)
• Subcadena: "es una prueba de"
• Rango : [0, 72057594037927936)

• Detalle por proceso
RANK | START | END | TESTS | STATUS | TIME(s)
-----|-----|-----|-----|-----|-----
0 | 0 | 18014398509481984 | 1 | STOP(SIGNAL) | 0.0012
1 | 18014398509481984 | 36028797018963968 | 1 | FOUND | 0.0010
2 | 36028797018963968 | 54043195528445952 | 1 | STOP(SIGNAL) | 0.0010
3 | 54043195528445952 | 72057594037927936 | 1 | STOP(SIGNAL) | 0.0010

• Resultado: ✓ Llave encontrada
- Rank : 1
- Llave : 18014398509481984 (efectiva dec=18014398509481984, hex=0x0040000000000000)
- Texto : Esta es una prueba de proyecto 2

• Resumen global
- Llaves probadas totales : 4
- Tiempo total (max rank) : 0.001151 s
and@AM-DELL:~/PY2-Paralela$

```

d)

En las pruebas realizadas se observa con claridad que el tiempo de ejecución depende directamente de la posición de la llave dentro del rango y de la forma en que MPI reparte ese rango entre los procesos.

Cuando se utilizó una llave pequeña (123456) dentro de un rango de 2^{24} , esta cayó al inicio del subrango asignado al proceso con rank 0. Como consecuencia, la llave se encontró en milésimas de segundo, ya que ese proceso la probó prácticamente de inmediato.

En el caso contrario, con la llave muy grande del escenario, correspondiente a $(2^{56}/4)-1$, el valor coincidió exactamente con el inicio del subrango asignado al rank 1. Allí ocurrió algo similar: el hallazgo fue instantáneo porque ese proceso la probó en su primer intento, mientras que los demás recibieron la señal y se detuvieron de inmediato.

Sin embargo, el escenario, con la llave $2^{56}/4 - 1$, resultó ser el peor caso. Este valor se ubicó al final del subrango asignado al rank 0, lo que obligó a ese proceso a recorrer prácticamente toda su porción ($\sim 2^{54}$ llaves) antes de encontrarla, volviendo inviable completar la búsqueda en pocos minutos.

En conclusión, bajo las mismas condiciones de hardware y algoritmo, el tiempo de ejecución crece con la “distancia” de la llave respecto al inicio del subrango asignado al proceso que la

contiene. Si la llave aparece justo al comienzo de un subrango, el tiempo de búsqueda es casi nulo; pero si cae al final, el tiempo se dispara.

3)

Alternativas:

Alternativa A Distribución cíclica

Idea: En lugar de asignar un bloque contiguo a cada rank, cada proceso prueba llaves intercaladas. Así, tras t iteraciones, el conjunto de llaves probadas cubre las primeras $t \cdot P$ llaves del rango, reduciendo la varianza y el sesgo por posición de la llave mejor tiempo esperado paralelo.

Alternativa B Master–Worker dinámico con chunks

Idea: Un maestro (rank 0) reparte ventanas de tamaño B a los workers según las van pidiendo. Así:

- Balancea mejor si algunos chunks terminan antes.
- Reduce el tiempo esperado ante posiciones adversas de la llave.
- Mantiene early-stop: cuando alguien encuentra, el maestro envía stop a todos.

Alternativa C permuted-stride

Idea: Cada proceso recorre índices intercalados de la forma $i = \text{rank} + t \cdot P$, pero antes de probar la llave aplica una permutación biyectiva del rango $[L, U)$ mediante un generador lineal congruencial (LCG) con la fórmula $\text{idx} = (A \cdot i + B) \bmod N$. Esto sirve para barajar el orden de prueba y evitar sesgos del orden secuencial. La detección de parada temprana se realiza punto a punto, sin operaciones colectivas dentro del bucle, evitando bloqueos entre procesos.

Alternativa D dinámico adaptativo

Idea: Un proceso maestro reparte bloques de llaves. Cada proceso trabajador solicita trabajo al maestro, quien estima su velocidad de procesamiento y ajusta el tamaño del bloque B para que dure aproximadamente T milisegundos, según el parámetro $-T$. Cuando algún proceso encuentra la llave correcta, el maestro envía una señal de stop a todos los demás. Finalmente, se imprimen las métricas de desempeño por cada proceso y se indica claramente cuál fue el ganador.

4)

Descripción / Pseudocódigo:

Naive (bloques contiguos + early-stop)

dividir $[L, U)$ en P bloques $[L_i, U_i)$

para cada rank i en paralelo:

 Irecv(found)

 para $k = L_i \dots U_i - 1$:

 si Test(Irecv) $\rightarrow$ parar (otro encontró)

 si match(k): found= k ; Send(found) a todos; parar

 fin

recolectar métricas en 0 y reportar

Alternativa A Cíclico

para cada rank i en paralelo:

 Irecv(found)

 para $k = L + i$; $k < U$; $k += P$:

 si Test(Irecv) $\rightarrow$ parar

 si match(k): found= k ; Send(found) a todos; parar

Reporte

Alternativa B Dinámico

MASTER (rank 0):

 next = L

 mientras no found y workers activos:

 si llega FOUND(k): found= k ; responde STOP a todos

 si llega REQ de w :

 si found o next $\geq U$: enviar STOP a w

 sino: enviar TASK[a =next, b =min(next+ B , U)]; next= b

WORKER (rank >0):

 loop:

 enviar REQ

 recibir TASK[a, b] o STOP

 si STOP $\rightarrow$ salir

 para k in [a, b):

 si match(k): enviar FOUND(k) a MASTER; salir

 si Iprobe(STOP) $\rightarrow$ recibir y salir

 reporte

Alternativa C permuted-stride

broadcast(cipher, needle, L , U)

$A \leftarrow$ impar coprimo con $N=U-L$

$B \leftarrow$ seed XOR $f(\text{rank})$

post Irecv(STOP)


```

t0 ← MPI_Wtime()
for t=0.. while not stop:
    if t % K == 0 and Irecv(STOP) completed: break
    i ← rank + t*P
    if i ≥ N: break
    idx ← (A*i + B) mod N
    key ← L + idx
    plain ← DES_decrypt(key, cipher)
    if contains(plain, needle):
        send STOP to all ranks
        winner ← rank; found_key ← key
        break
gather(times, tests, found_flags, keys)
winner ← root picks fastest FOUND
broadcast(winner, found_key)
print tabla por proceso y resultado

```

Alternativa D dinámico adaptativo

```

broadcast(cipher, needle, L, U)
if rank==0 (maestro):
    init next←L, end←U, B[w]≈B0
    para cada worker: recibir REQ y enviar TASK=[a,b) con b-a=B[w]
    loop:
        Probe cualquier mensaje:
        si FOUND(k) desde w*: broadcast STOP, winner←w*, found_key←k, break
        si REQ desde w:
            actualizar throughput[w] con (B[w]/tiempo_chunk)
            B[w] ← clamp(throughput[w] * (T_ms/1000), [Bmin,Bmax])
            asignar TASK siguiente [next, next+B[w])
recolectar (tiempos, tests, status) con Gather
root imprime tabla y resultado (marcando winner)

```

Referencias:

- Msmk. (2024, 10 septiembre). *¿Qué es el DES?* MSMK.
<https://msmk.university/que-es-el-des-msmk-university/>
- *IBM Semeru Runtime Certified Edition for z/OS*. (2025).
<https://www.ibm.com/docs/es/semeru-runtime-ce-z/21.0.0?topic=differences-data-encryption-standard-des-keys-operations>